

UNIVERSIDAD POLITÉCNICA DE CHIAPAS
MAESTRÍA EN CIENCIAS E INNOVACIÓN TECNOLÓGICA

ASIGNATURA
LÓGICA Y PROGRAMACIÓN

EVALUACIÓN 04 - Informe individual

Alumna
Carmen Anel Vilca Mamani

DOCENTE
DR. HORACIO IRÁN SOLÍS CISNEROS

Tuxtla Gutiérrez, Chiapas
13 de Marzo de 2026

1. Introducción

El proyecto desarrollado en equipo tuvo como propósito evaluar el rendimiento de un sistema de sincronización de semáforos mediante la comparación de distintas configuraciones de ejecución. Para ello, el sistema fue analizado a través de un pipeline de procesamiento, cuya implementación secuencial y concurrente fue comparada experimentalmente con el fin de identificar diferencias en tiempo de ejecución, speedup y capacidad de procesamiento, especialmente cuando el tamaño del problema aumenta.

Dentro de este trabajo colectivo, el presente informe corresponde al aporte individual orientado a la visualización y análisis de resultados experimentales. Esta participación se enfocó en transformar los datos numéricos obtenidos previamente en representaciones gráficas claras, útiles para interpretar el comportamiento del sistema y para apoyar la discusión sobre su rendimiento y escalabilidad.

Por ello, este informe no se centra en la construcción interna del sistema ni en la etapa original de benchmarking, sino en la lectura de los datos experimentales, el cálculo de métricas derivadas, la generación de gráficas comparativas y la interpretación de la evidencia obtenida. Desde esta perspectiva, el documento busca aportar claridad, coherencia y sustento visual al análisis del proyecto.

2. Objetivo del aporte individual

El objetivo de este aporte individual fue generar visualizaciones comparativas a partir de los resultados experimentales del sistema, con la finalidad de facilitar su interpretación y discusión. De manera más específica, se buscó representar gráficamente la mejora relativa del rendimiento, la comparación de tiempos de ejecución y la capacidad de procesamiento observada en cada configuración evaluada.

Asimismo, se pretendió que estas visualizaciones permitieran identificar tendencias de comportamiento conforme aumenta el número de intersecciones procesadas, de modo que fuera posible discutir con mayor claridad el rendimiento alcanzado por la implementación secuencial y por las variantes concurrentes, así como la escalabilidad del sistema.

3. Descripción de los datos utilizados y procedimiento de visualización

Para la elaboración de este aporte se utilizó el archivo `results.csv`, el cual contenía los resultados resumidos del proceso de benchmarking desarrollado previamente por otro integrante del equipo. En la versión final del script, este archivo no se ubicó en la misma carpeta del código de visualización, sino dentro del directorio `benchmarks`, por lo que fue leído mediante una ruta relativa construida desde la ubicación del propio script. En términos prácticos, el archivo fue cargado desde:

```
../benchmarks/results.csv
```

Dicho archivo incluyó información correspondiente a distintos tamaños de problema, expresados mediante el número de intersecciones procesadas. Los valores considerados fueron $n = 10, 25, 50, 100$ y 200 .

Las columnas empleadas para el análisis y la visualización fueron las siguientes:

- **n**: número de intersecciones procesadas.
- **t_p**: tiempo de ejecución de la versión secuencial.
- **t_q_async**: tiempo de ejecución de la versión concurrente con un worker.
- **t_q_full**: tiempo de ejecución de la versión concurrente con cuatro workers.
- **speedup_async**: mejora relativa de la configuración concurrente con un worker respecto a la secuencial.
- **speedup_full**: mejora relativa de la configuración concurrente con cuatro workers respecto a la secuencial.
- **efficiency**: eficiencia observada en la configuración concurrente.

A partir de estos datos se generaron tres gráficas con la biblioteca `matplotlib` en Python:

1. **Speedup vs n**
2. **Comparación de tiempos de ejecución**
3. **Throughput vs n**

En el caso del throughput, esta métrica no se encontraba originalmente como una columna del archivo CSV, por lo que fue necesario calcularla durante el procedimiento de visualización. Para ello se utilizó la siguiente expresión:

$$\text{Throughput} = \frac{n}{t}$$

donde n representa el número de intersecciones procesadas y t el tiempo de ejecución asociado a una configuración determinada. A partir de esta fórmula se calcularon las columnas `throughput_p`, `throughput_q_async` y `throughput_q_full`, correspondientes a la implementación secuencial, a la versión concurrente con un worker y a la versión concurrente con cuatro workers, respectivamente.

Como parte del procedimiento, primero se verificó la lectura correcta del archivo `results.csv` y la presencia de las columnas necesarias para el análisis. Después, el script imprimió en consola la tabla original del CSV y, posteriormente, la misma tabla con las nuevas columnas de throughput ya incorporadas. Esta comprobación fue útil para confirmar que el cálculo se realizó correctamente antes de generar las gráficas finales.

La Figura 1 muestra la salida en consola del procedimiento, donde puede observarse tanto la tabla original del archivo CSV como la incorporación de las columnas derivadas del cálculo de throughput.

csv original										
	n	t_p	t_q_async	t_q_full	speedup_async	speedup_full	efficiency			
0	10	0.568	0.086	0.056	6.604651	10.142857	0.383929			
1	25	1.421	0.158	0.075	8.993671	18.946667	0.526667			
2	50	2.831	0.265	0.106	10.683019	26.707547	0.625000			
3	100	5.926	0.488	0.170	12.143443	34.858824	0.717647			
4	200	12.153	0.926	0.295	13.124190	41.196610	0.784746			
Añadiendo las columnas con los calculos del throughput										
	n	t_p	t_q_async	t_q_full	speedup_async	speedup_full	efficiency	throughput_p	throughput_q_async	throughput_q_full
0	10	0.568	0.086	0.056	6.604651	10.142857	0.383929	17.605634	116.279070	178.571429
1	25	1.421	0.158	0.075	8.993671	18.946667	0.526667	17.593244	158.227848	333.333333
2	50	2.831	0.265	0.106	10.683019	26.707547	0.625000	17.661604	188.679245	471.698113
3	100	5.926	0.488	0.170	12.143443	34.858824	0.717647	16.874789	204.918033	588.235294
4	200	12.153	0.926	0.295	13.124190	41.196610	0.784746	16.456842	215.982721	677.966102

Figura 1: Lectura del archivo `results.csv` y cálculo de las columnas de throughput mediante el script de visualización.

Una vez verificada la lectura de datos y realizado el cálculo del throughput, el script generó automáticamente las tres gráficas requeridas. En la versión final del código, las figuras se guardaron en la misma carpeta donde se encuentra el script de visualización, lo cual permitió mantener organizados en un mismo directorio el archivo `generar_graficas.py` y los archivos de salida `speedup_vs_n.png`, `tiempos_comparativa.png` y `throughput_vs_n.png`. Todas las figuras fueron elaboradas con título, ejes etiquetados y leyenda, con el propósito de facilitar su lectura, asegurar claridad en la presentación y cumplir con los criterios de calidad del manuscrito.

4. Resultados

Los resultados obtenidos se presentan mediante tres gráficas comparativas que permiten observar con claridad las diferencias entre la implementación secuencial y las variantes concurrentes del sistema. Estas visualizaciones no solo muestran los valores alcanzados en cada configuración, sino que también permiten identificar tendencias generales de comportamiento conforme aumenta el número de intersecciones procesadas.

Las figuras revelan que el sistema concurrente mantiene un desempeño más favorable que la versión secuencial en todos los tamaños de problema evaluados, esta diferencia se aprecia desde distintos enfoques: la mejora relativa del rendimiento, la reducción de tiempos de ejecución y el incremento de la capacidad de procesamiento medida en intersecciones por segundo. De esta manera, la presentación gráfica de los resultados permite comparar métodos de forma directa y aporta una base clara para el análisis posterior.

Además, el conjunto de visualizaciones permite apreciar que los resultados no muestran una ventaja aislada o puntual, sino un patrón estable a lo largo de todos los valores de n estudiados. Esto es importante porque da mayor solidez a la interpretación general del experimento: la propuesta concurrente no mejora solo en un caso particular, sino que mantiene un comportamiento superior en distintos tamaños del problema.

4.1. Speedup vs n

La primera gráfica muestra cómo evoluciona el speedup conforme aumenta el tamaño del problema. En esta representación se comparan dos variantes de la arquitectura concurrente: *AsyncIO only* y *AsyncIO+MP*. La figura permite observar que ambas configuraciones superan a la implementación secuencial, pero también evidencia que la variante que combina asincronía con multiprocesamiento alcanza valores de mejora considerablemente más altos.

Además, la gráfica muestra que la diferencia entre ambas configuraciones concurrentes se vuelve más evidente a medida que crece el valor de n . Esto sugiere que la ventaja del enfoque más completo no solo se mantiene, sino que se fortalece cuando la carga de trabajo es mayor. Por ello, esta visualización constituye una primera evidencia de que la arquitectura concurrente ofrece una mejora real de rendimiento y de que dicha mejora es especialmente significativa en la configuración con varios workers.

También resulta relevante observar que la curva correspondiente a *AsyncIO only* no permanece plana, sino que mantiene una tendencia ascendente. Esto indica que incluso la versión concurrente más simple aporta beneficios sostenidos frente a la implementación secuencial. Sin embargo, la separación progresiva con respecto a *AsyncIO+MP* muestra que la incorporación de múltiples workers añade una ganancia adicional importante, especialmente en escenarios de mayor carga.

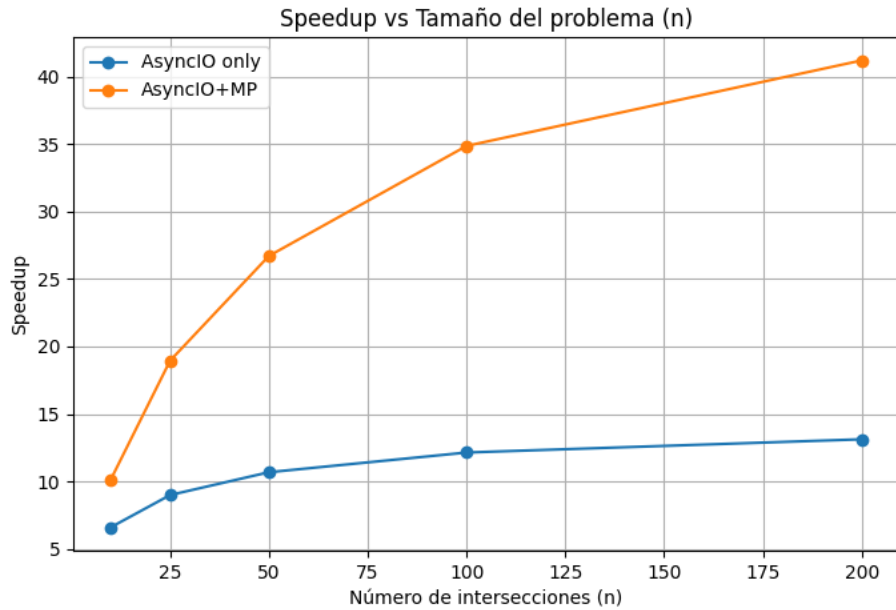


Figura 2: Speedup en función del número de intersecciones.

4.2. Comparación de tiempos de ejecución

La segunda gráfica presenta una comparación directa de los tiempos de ejecución entre las tres configuraciones evaluadas: la versión secuencial (P), la versión concurrente con un worker (Q -*async*) y la versión concurrente con cuatro workers (Q -*full*). Esta representación

resulta útil porque permite identificar de forma inmediata cuál configuración requiere más tiempo para procesar el mismo volumen de trabajo.

A partir de la figura puede apreciarse que la implementación secuencial presenta los tiempos más altos en todos los tamaños de problema considerados. En cambio, las variantes concurrentes mantienen tiempos mucho menores, siendo la configuración con cuatro workers la que presenta los mejores resultados. La diferencia visual entre las barras se acentúa conforme crece el número de intersecciones, lo que pone de manifiesto que la arquitectura secuencial pierde eficiencia con mayor rapidez, mientras que las configuraciones concurrentes responden de forma más favorable.

Esta gráfica ofrece una evidencia directa del beneficio práctico de la concurrencia, no se trata solo de una mejora teórica, sino de una reducción observable del tiempo de ejecución en todos los escenarios evaluados. A medida que el problema crece, la distancia entre la barra secuencial y las barras concurrentes se vuelve más amplia, lo que refuerza la idea de que el enfoque concurrente resulta más adecuado para escenarios en los que la carga de trabajo aumenta.

Asimismo, la diferencia entre *Q-async* y *Q-full* permite advertir que no todas las configuraciones concurrentes producen el mismo nivel de mejora, aunque ambas superan a la implementación secuencial, la versión con cuatro workers mantiene una ventaja visible en todos los casos, confirmando que el paralelismo adicional repercute directamente en una ejecución más eficiente.

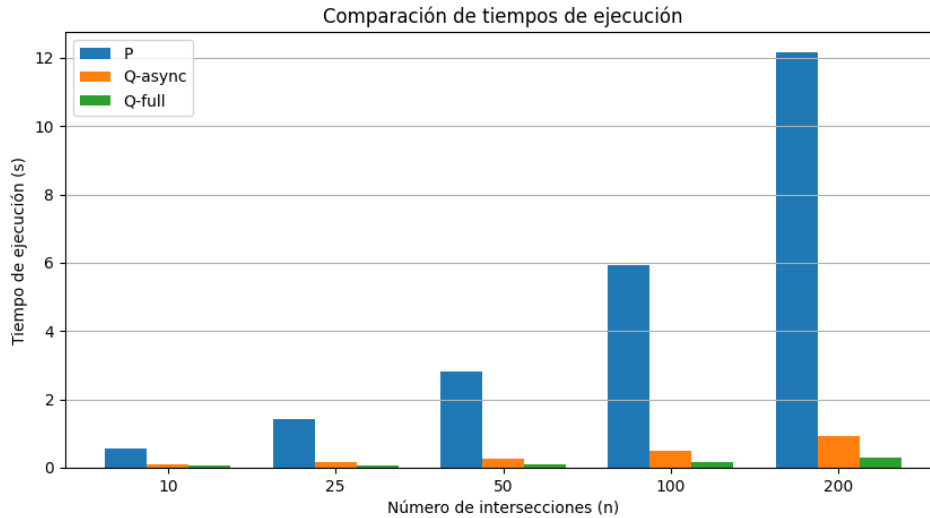


Figura 3: Comparación de tiempos de ejecución entre las distintas configuraciones.

4.3. Throughput vs n

La tercera gráfica representa el throughput del sistema, entendido como la cantidad de intersecciones procesadas por segundo. Esta visualización complementa a la comparación de tiempos, ya que permite observar el rendimiento desde una perspectiva de productividad computacional. Mientras los tiempos muestran cuánto tarda cada configuración, el through-

put permite apreciar cuánto trabajo logra completar cada una en una unidad de tiempo.

En la figura se observa que la implementación secuencial mantiene los valores más bajos de throughput en todos los casos. Por su parte, la configuración concurrente con un worker alcanza un desempeño intermedio, claramente superior al secuencial. Sin embargo, la configuración con cuatro workers destaca con valores mucho mayores, lo que indica una capacidad de procesamiento significativamente más alta.

También puede advertirse que, a medida que aumenta el tamaño del problema, las configuraciones concurrentes incrementan su throughput de forma más favorable que la implementación secuencial. Esto refuerza la idea de que el sistema concurrente no solo reduce tiempos, sino que además aprovecha mejor los recursos disponibles para procesar una mayor cantidad de trabajo por segundo.

Desde una lectura más amplia, esta gráfica resulta especialmente valiosa porque no se limita a mostrar cuánto se ahorra en tiempo, sino cuánto se gana en capacidad efectiva de procesamiento. Por ello, el throughput actúa como una métrica complementaria que fortalece la interpretación del rendimiento global del sistema.

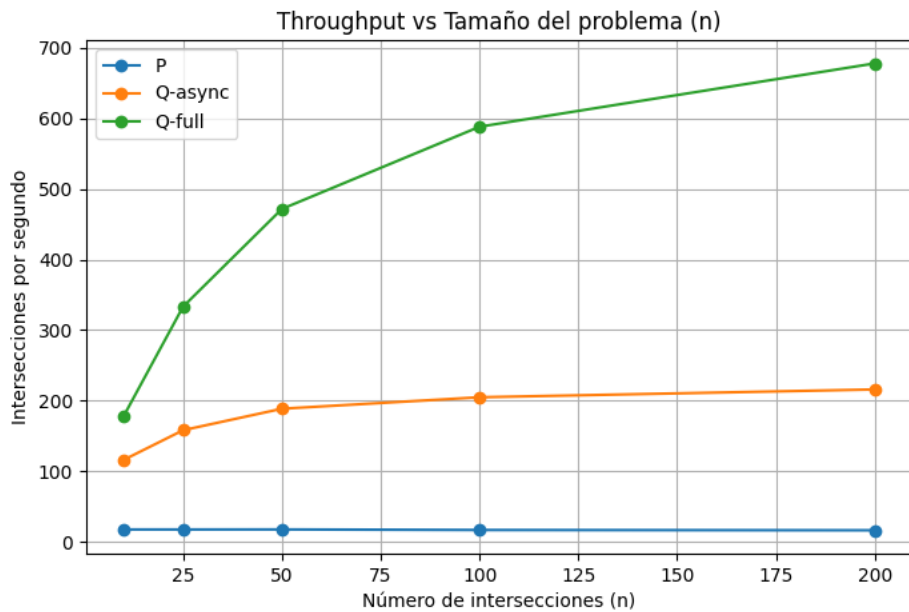


Figura 4: Throughput del sistema en función del número de intersecciones.

5. Análisis y discusión

A partir de las gráficas obtenidas puede afirmarse que los resultados experimentales muestran un patrón consistente a favor de la arquitectura concurrente. En primer lugar, la implementación secuencial presenta siempre los peores valores de desempeño, tanto en tiempo de ejecución como en throughput, esto indica que su capacidad para atender la carga de trabajo del sistema resulta más limitada, especialmente cuando el número de intersecciones aumenta.

En segundo lugar, la configuración concurrente con un worker ya ofrece una mejora importante respecto a la ejecución secuencial, este resultado es relevante porque sugiere que la ventaja de la propuesta no depende exclusivamente del uso intensivo de múltiples workers, sino también de una organización más eficiente del procesamiento. En otras palabras, el rediseño de la arquitectura aporta beneficios incluso antes de explotar completamente el paralelismo.

No obstante, la configuración con cuatro workers es la que presenta el mejor comportamiento global. La gráfica de speedup muestra que la mejora relativa obtenida por *AsyncIO+MP* es considerablemente mayor que la alcanzada por *AsyncIO only*. De forma complementaria, la gráfica de tiempos evidencia que esta configuración reduce de manera más pronunciada la duración de la ejecución, mientras que la gráfica de throughput confirma que también es la que procesa más intersecciones por segundo. La coincidencia entre estas tres perspectivas fortalece la validez del análisis, ya que todas apuntan en la misma dirección.

Desde el punto de vista interpretativo, estos resultados pueden relacionarse con la teoría general de concurrencia y paralelismo, cuando el trabajo se distribuye de forma más adecuada y se aprovechan mejor los recursos computacionales, es posible reducir cuellos de botella y mejorar el rendimiento global del sistema. En este caso, la evidencia visual obtenida respalda precisamente esa idea: la arquitectura concurrente, y particularmente la variante con varios workers, logra una ejecución más eficiente que la implementación secuencial.

Asimismo, los resultados comparan de manera clara distintos métodos de ejecución, lo que permite una discusión más sólida del comportamiento del sistema, aunque este informe se basa en resultados ya resumidos del benchmark original, las gráficas ofrecen una visión clara de las tendencias generales como para sostener una interpretación coherente del rendimiento observado.

También es importante señalar que la concordancia entre las distintas métricas fortalece la consistencia del análisis. El speedup, los tiempos de ejecución y el throughput no muestran señales contradictorias entre sí; por el contrario, las tres representaciones conducen a la misma conclusión general: la arquitectura concurrente mejora el rendimiento, y la configuración con cuatro workers es la que ofrece el comportamiento más favorable.

6. Escalabilidad del sistema

La escalabilidad se refiere a la capacidad de un sistema para mantener un comportamiento eficiente conforme aumenta la carga de trabajo, en este proyecto, dicha carga está representada por el número de intersecciones procesadas. Desde esta perspectiva, un sistema escalable no solo debe seguir funcionando cuando el problema crece, sino hacerlo sin que su rendimiento se degrade de manera desproporcionada.

Las gráficas obtenidas permiten observar que la implementación secuencial presenta una escalabilidad limitada, conforme aumenta n , sus tiempos de ejecución crecen de manera marcada y su throughput permanece bajo en comparación con el de las configuraciones concurrentes. Este comportamiento indica que la versión secuencial enfrenta mayores dificultades para adaptarse al incremento de la carga de trabajo.

Por otro lado, la versión concurrente con un worker ya muestra una respuesta más favorable, sus tiempos son menores y su throughput es claramente superior al de la implementación

secuencial. Sin embargo, la mejor evidencia de escalabilidad se encuentra en la configuración con cuatro workers, ya que mantiene tiempos considerablemente más bajos y alcanza los mayores valores de throughput en todos los escenarios evaluados.

Además, la gráfica de speedup muestra que la ventaja relativa de la arquitectura concurrente no desaparece cuando el tamaño del problema aumenta, por el contrario, dicha ventaja se vuelve más evidente conforme crece n . Esto sugiere que el sistema concurrente aprovecha de mejor manera los recursos disponibles y responde con mayor eficiencia ante problemas de mayor tamaño.

En consecuencia, puede concluirse que la arquitectura concurrente implementada en el proyecto presenta una mejor escalabilidad que la secuencial, especialmente en la configuración que combina asincronía y multiprocesamiento.

7. Limitaciones

Aunque las gráficas generadas permiten identificar tendencias claras y comparar el comportamiento de distintas configuraciones, el presente análisis presenta algunas limitaciones. En primer lugar, este aporte se basó en los resultados resumidos contenidos en el archivo `results.csv`, por lo que la interpretación se realizó sobre valores ya procesados y no sobre el conjunto completo de ejecuciones individuales del benchmark.

Asimismo, aunque las figuras muestran con claridad las diferencias entre métodos, en este informe no se incorporaron medidas explícitas de variabilidad, como barras de error, intervalos o desviación estándar dentro de las gráficas, por ello, el análisis se centra principalmente en tendencias generales y en los valores promedio reportados previamente.

Otra limitación importante es que los resultados dependen del entorno experimental en que fueron obtenidos, es decir, factores como el hardware utilizado, la carga del procesador o las condiciones específicas de ejecución pueden influir en el comportamiento observado. Por esta razón, las conclusiones deben entenderse dentro del contexto experimental del proyecto.

Como posible trabajo futuro, sería conveniente complementar este análisis con medidas de dispersión, tamaños de problema adicionales y nuevas configuraciones de workers, esto permitiría fortalecer la validez del estudio y profundizar en el análisis de la escalabilidad del sistema.

8. Conclusiones

El presente aporte individual permitió transformar los resultados experimentales del benchmarking en representaciones gráficas claras y comparables, útiles para apoyar la interpretación del rendimiento del sistema. La lectura del archivo `results.csv`, el cálculo del throughput y la generación de las visualizaciones hicieron posible contar con una base gráfica sólida para el análisis del proyecto.

Los resultados mostraron de manera consistente que la arquitectura concurrente supera a la implementación secuencial tanto en reducción de tiempos de ejecución como en capacidad de procesamiento. Asimismo, la gráfica de speedup evidenció que la mejora relativa aumenta conforme crece el tamaño del problema, especialmente en la configuración que combina asincronía y multiprocesamiento.

Desde la perspectiva de escalabilidad, las figuras obtenidas permiten sostener que la versión concurrente responde de manera más eficiente ante el incremento de la carga de trabajo, siendo la configuración con cuatro workers la que presenta el mejor comportamiento general.

Considero que este informe no solo presenta resultados mediante figuras correctamente etiquetadas, sino que también desarrolla un análisis claro, coherente y profundo de la evidencia experimental, por ello, las visualizaciones elaboradas constituyen un apoyo importante para sustentar que la propuesta concurrente del sistema de sincronización de semáforos ofrece una mejora real de rendimiento frente a la implementación secuencial.